

DSA STUDY BLUEPRINT SERIES

26. String Compression Leetcode 443

String Compression and Run-Length Encoding

Section 04 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Compress character duplicates to character + frequency.
- Write compression results in-place to avoid auxiliary arrays.

Memory & Execution Blueprint

`[a, a, a, b, b, c] ⇒ [a, 3, b, 2, c]`

C++ Implementation Reference

Standard code layout and syntax

```
int compress(vector& chars) {  
    int writeIdx = 0, i = 0;  
    while (i < chars.size()) {  
        char curr = chars[i];  
        int count = 0;  
        while (i < chars.size() && chars[i] == curr) { i++; count++; }  
        chars[writeIdx++] = curr;  
        if (count > 1) {  
            for (char c : to_string(count)) chars[writeIdx++] = c;  
        }  
    }  
    return writeIdx;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(1)$ (In-place modification)

Key Practices & Gotchas

- **Important:** Only write frequency count value if it is strictly greater than 1.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run two-pointer string validations or window updates for string "race" :

Step	Left Pointer Index	Right Pointer Index	Char Comparison	Outcome
1	left = 0 ('r')	right = 3 ('e')	'r' != 'e'	Validation stops (Not a palindrome)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Sliding Window Patterns:** Track character frequency maps within a moving boundary window size to find substring anagrams and permutations.
- **In-place reversals:** Swap characters inside word boundaries using double pointer offsets.
- **Related LeetCode Problems:** #3 Longest Substring Without Repeating Characters, #567 Permutation in String, #344 Reverse String.